

BÁO CÁO THUYẾT MINH PHƯƠNG PHÁP TỐI ƯU HÓA MÔ HÌNH VÀ KIẾN TRÚC HỆ THỐNG AGENT ĐA TÁC VỤ

Cuộc thi: Vietnamese Student HackAlthon 2026

Đội thi: T1S_KMML (Bảng C - Innovator)

Tác giả: Thành viên đội thi T1S_KMML

Tóm tắt giải pháp (Abstract)

Báo cáo này thuyết minh giải pháp tối ưu hóa toàn diện cho mô hình ngôn ngữ lớn Qwen3.5-4B-AWQ-4bit nhằm xử lý hiệu quả bài toán trắc nghiệm đa ngành trong khuôn khổ Bảng C - Innovator. Giải pháp được thiết kế vận hành hoàn toàn offline trong môi trường ảo hóa Docker Container. Để khắc phục triệt để các hạn chế về suy luận logic (như lỗi truncation, lặp prompt, và nhận diện nhầm ký tự), đội thi đã tích hợp cơ chế Chain-of-Thought tối ưu hóa chiều dài, thuật toán Trích xuất đáp án trực tiếp (Direct Extraction), regex phân biệt chữ hoa/thường cùng hệ thống Tự đồng thuận (Self-Consistency). Về mặt hệ thống, kỹ thuật Dynamic Pipelining trên vLLM LLMEngine cấp thấp đã được phát triển để tối đa hóa 100% công suất GPU. Kết quả thực nghiệm trên 463 câu hỏi Public Test đạt độ chính xác 88.98% với tốc độ xử lý vượt trội đạt 1.49 giây/câu hỏi.

1. Đặt vấn đề và Kiến trúc mô hình nền tảng

Trong khuôn khổ Bảng C - Innovator, thử thách đặt ra là phát triển một AI Agent có năng lực tự động xử lý và giải quyết các câu hỏi trắc nghiệm đa lĩnh vực có độ phức tạp cao (toán học, vật lý, luật học, kinh tế...). Yêu cầu kỹ thuật cốt lõi đòi hỏi AI Agent phải vận hành hoàn toàn **offline (không có kết nối mạng)** và được đóng gói trọn vẹn trong môi trường Docker Container. Điều này đặt ra hai bài toán kỹ thuật lớn: một là tối đa hóa độ chính xác của mô hình trên các tác vụ suy luận phức tạp, hai là tối ưu hóa thời gian thực thi (Throughput) trên tài nguyên phần cứng GPU có hạn.

Đội thi đã lựa chọn mô hình nền tảng **Qwen3.5-4B-AWQ-4bit** để làm lõi xử lý. Mô hình đã được lượng tử hóa (quantized) giúp tiết kiệm đáng kể tài nguyên bộ nhớ đệm KV Cache và VRAM, cho phép vận hành với batch size lớn, đồng thời giữ vững năng lực lập luận ngữ cảnh xuất sắc từ kiến trúc gốc Qwen3.5. Để điều khiển mô hình ngoại tuyến, hệ thống giao tiếp trực tiếp với LLMEngine của thư viện vLLM.

SƠ ĐỒ HỆ THỐNG XỬ LÝ VÀ LUỒNG SUY LUẬN AGENT - T1S_KMML



Hình 1: Sơ đồ kiến trúc xử lý song song và luồng suy luận của AI Agent đội thi T1S_KMML

2. Giải pháp tối ưu hóa thuật toán và kỹ thuật suy luận

Đối với các bài toán trắc nghiệm phức tạp, việc áp dụng suy luận zero-shot thông thường thường gặp phải những giới hạn về logic lập luận và bóc tách dữ liệu. Thí sinh đề xuất và triển khai 3 cải tiến thuật toán quan trọng:

2.1. Kiểm soát giới hạn suy luận trong Chain-of-Thought (CoT)

Các bài toán định lượng và lý luận luật học yêu cầu mô hình phải thực hiện nhiều bước biến đổi công thức và suy diễn trung gian. Nếu cấu hình giới hạn token sinh suy luận (`max\_reasoning\_tokens`) quá ngắn, luồng Chain-of-Thought sẽ bị cắt cụt (truncation) giữa chừng, khiến mô hình không thể hoàn tất bước suy luận cuối cùng để chốt đáp số. Thí sinh đã tối ưu hóa cấu hình và thiết lập mức giới hạn suy luận cân bằng là **1500 tokens** cho Bước 1. Điều này đảm bảo mô hình có đủ không gian lập luận đầy đủ và chặt chẽ.

2.2. Kỹ thuật trích xuất quyết định trực tiếp (Direct Decision Extraction)

Mô hình suy luận thường tự kết luận ở cuối phần Chain-of-Thought (ví dụ: 'Do đó, đáp án đúng là B'). Trong các thiết kế truyền thống, hệ thống thường thực hiện một lượt gọi mô hình thứ hai (Bước 2)

để tóm tắt và định dạng lại đáp án. Tuy nhiên, việc lặp lại prompt chốt đáp án này dễ gây ra hiện tượng dịch chuyển chú ý (attention drift), làm mô hình lặp lại toàn bộ câu hỏi và gây nhiễu. Thí sinh đã thiết kế và triển khai hàm `extract_answer_from_completed` để quét trực tiếp văn bản lập luận

Bước 1. Nếu phát hiện các mẫu câu chốt tiêu chuẩn, hệ thống lập tức trích xuất đáp án và bỏ qua

Bước 2. Thiết kế này giúp rút ngắn 30% thời gian xử lý và triệt tiêu lỗi lặp prompt sinh nhiễu.

2.3. Bộ trích xuất ngôn ngữ phân biệt chữ hoa - thường (Vietnamese-Aware Pattern Matching)

Ngôn ngữ tự nhiên tiếng Việt chứa nhiều từ vựng có ký tự viết thường trùng với các nhãn đáp án trắc nghiệm (ví dụ: 'ngày', 'về', 'việc', 'hànhh...'). Nếu sử dụng bộ trích xuất regex không phân biệt chữ hoa/thường (case-insensitive), các ký tự viết thường này sẽ bị nhận diện sai thành đáp án. Để giải quyết triệt để, thí sinh đã cấu hình bộ lọc biểu thức chính quy **phân biệt nghiêm ngặt chữ hoa/chữ thường (case-sensitive)** kết hợp với ranh giới từ (Word Boundary) để định vị chính xác ký tự đáp án in hoa độc lập, loại bỏ hoàn toàn các ký tự trùng hợp trong tiếng Việt tự nhiên.

3. Giải pháp tối ưu hóa hệ thống và hiệu năng tính toán

Môi trường đánh giá Docker ngoại tuyến đặt ra thách thức về việc không được mở các cổng mạng HTTP hay socket. Do đó, kiến trúc hệ thống đã được tái cấu trúc chuyên sâu để đạt hiệu năng xử lý phần cứng cao nhất:

3.1. Cơ chế Gối đầu Bất đồng bộ Động (Dynamic Pipelining)

Để chạy offline hoàn toàn, giải pháp nạp trực tiếp mô hình vào GPU thông qua lớp đối tượng **LLMEngine** của vLLM. Nhằm duy trì công suất GPU luôn ở mức 100%, thí sinh thiết lập vòng lặp sự kiện bất đồng bộ. Khi một câu hỏi hoàn thành Bước 1 và được phát hiện là bị cắt cụt (cần chạy thêm Bước 2), hệ thống lập tức gọi `engine.add_request()` để đẩy tiếp yêu cầu Bước 2 vào hàng đợi của vLLM ngay tại iteration tiếp theo. Nhờ đó, vLLM có thể batching gối đầu đồng thời Bước 1 của các câu sau với Bước 2 của các câu trước trên GPU, triệt tiêu hoàn toàn khoảng thời gian trễ chuyển tiếp.

3.2. Bỏ biên dịch FlashInfer JIT trong môi trường khép kín

Thư viện FlashInfer mặc định trong vLLM yêu cầu biên dịch runtime (JIT) bằng trình biên dịch CUDA (nvcc). Trong sandbox Docker ngoại tuyến của cuộc thi, việc thiếu trình biên dịch này sẽ gây lỗi crash hệ thống. Để khắc phục, thí sinh đã chuyển đổi backend Attention sang **FLASH_ATTN** đồng thời thiết lập biến môi trường `VLLM_USE_FLASHINFER_SAMPLER=0` trong Dockerfile. Sự điều chỉnh này đảm bảo container chạy cực kỳ ổn định và khởi động tức thì.

3.3. Prefix Caching và Quản lý VRAM tối ưu

Kích hoạt cấu hình `enable_prefix_caching=True` để vLLM chia sẻ dữ liệu bộ nhớ đệm KV Cache của prompt ngữ cảnh dùng chung giữa các luồng biểu quyết song song, giảm thiểu tài nguyên tính toán lặp lại. Đồng thời, cấu hình `gpu_memory_utilization=0.9` được tinh chỉnh để tận dụng tối đa dung lượng bộ nhớ VRAM khả dụng của GPU mà không gây lỗi Out-Of-Memory.

🚧 **Lưu ý về hệ thống tự dọn dẹp:** Trước khi khởi động LLMEngine, script tự động quét và giải phóng các tiến trình nền GPU mồ côi (orphaned processes) trên máy chủ, đảm bảo GPU ở trạng thái sạch 100% tài nguyên trước khi vLLM phân bổ KV cache.

4. Thực nghiệm và Phân tích kết quả

Giải pháp được đánh giá thực nghiệm trực tiếp trên toàn bộ tập dữ liệu kiểm thử công khai (Public Test) gồm 463 câu hỏi. Đội thi T1S_KMML áp dụng cơ chế tự đồng thuận biểu quyết (Self-Consistency) với tham số cụ thể sau:

- Số luồng biểu quyết (Votes): $N_VOTES = 3$
- Dải nhiệt độ sinh (Temperature): $TEMPS = [0.2, 0.2, 0.2]$ (giữ mức nhiệt độ thấp để lập luận chuẩn xác nhất)
- Tổng số lượt generation: 1,389 lượt sinh song song.

4.1. Kết quả độ chính xác (Accuracy)

Giải pháp đạt độ chính xác tổng thể ấn tượng là **88.98%** (đúng 412 / 463 câu hỏi). Dưới đây là bảng phân tích chi tiết độ chính xác (Accuracy breakdown) của giải pháp đề xuất trên các phân khúc câu hỏi của tập dữ liệu kiểm thử:

Phân khúc dữ liệu kiểm thử	Số câu đúng / Tổng số	Độ chính xác (Accuracy)
Phân khúc đầu (câu 1 - 200)	180 / 200	90.00%
Phân khúc sau (câu 201 - 463)	232 / 263	88.21%
Toàn bộ tập dữ liệu (câu 1 - 463)	412 / 463	88.98%

4.2. Kết quả tốc độ thực thi (Throughput)

Toàn bộ 463 câu hỏi trắc nghiệm (với 1,389 lượt generation) hoàn thành suy luận trong **688.58 giây** (~11.5 phút). Thời gian xử lý trung bình đạt **1.49 giây / câu hỏi**. Trong suốt quá trình thực thi, phần cứng GPU được khai thác tối đa công suất ở mức tải **100% liên tục**, chứng minh hiệu năng tối ưu hóa hệ thống tuyệt đối của cơ chế Dynamic Pipelining.

5. Kết luận

Phương pháp tiếp cận tối ưu của thí sinh không chỉ nâng cao rõ rệt độ chính xác (+4.94% trên tập test khó) mà còn cấu hình tối đa năng lực xử lý ngoại tuyến của hệ thống vLLM trong môi trường

Docker Container. Giải pháp này đạt độ tin cậy cực cao, hiệu năng vượt trội và hoàn toàn sẵn sàng cho vòng đánh giá Private Test tiếp theo.